

Jet algorithms used in the FCC jet benchmarks

FCCJetBenchmarks

September 13, 2026

All jets are built with FastJet through the `FCCAnalyses JetClustering::clustering_*` functors, from `src/histmaker_tools/jets.py`. Every algorithm below is a sequential recombination algorithm: repeatedly find the smallest of all d_{ij} and d_{iB} , merge the pair ij (or, if a d_{iB} is smallest, declare i a jet / recombine it with the beam), and repeat. They differ only in the distance measure and in the termination rule.

Unless stated otherwise the recombination scheme is the E -scheme (four-momenta are simply added, `recombination` = 0), and the input is the collection of visible particles with $|\eta| < 2.56$: stable generator particles with neutrinos removed for gen jets, particle-flow objects for reco jets.

The e^+e^- distance measures

The generalized e^+e^- k_T family (FastJet `ee_genkt_algorithm`) uses angular, not rapidity, distances, which is the natural choice at a lepton collider where there is no beam remnant to be boosted along:

$$d_{ij} = \min(E_i^{2p}, E_j^{2p}) \frac{1 - \cos \theta_{ij}}{1 - \cos R}, \quad d_{iB} = E_i^{2p}. \quad (1)$$

The exponent p selects the algorithm:

$$p = -1 \text{ (anti-}k_T\text{)}, \quad p = 0 \text{ (Cambridge/Aachen)}, \quad p = +1 \text{ (}k_T\text{)}.$$

All three are run in these benchmarks, inclusively, over the same radius grid. Two properties of Eq. (1) matter for reading the figures:

- The factor $1/(1 - \cos R)$ is a *global constant*: it never reorders the d_{ij} among themselves. The entire R dependence therefore enters through the comparison of d_{ij} against d_{iB} , i.e. through which particles are split off into their own jet rather than merged. Small R means more, narrower jets.
- As $R \rightarrow \pi$ the beam term can no longer win, and e^+e^- generalized k_T with $p = +1$ degenerates into the Durham algorithm below. Measured on simulated 6-jet events, exclusive-to- N clustering with $p = +1$ is already *bit-identical* to Durham for $R \gtrsim 0.6$. This is why the k_T family in these benchmarks is run *inclusively*, where R genuinely changes the jets.

Algorithms appearing in the figures

Durham (e^+e^- k_T , **exclusive**). FastJet `ee_kt_algorithm`; the classic LEP jet definition. It is Eq. (1) with $p = +1$ and no R at all, i.e. a pure

$$y_{ij} = \frac{2 \min(E_i^2, E_j^2) (1 - \cos \theta_{ij})}{E_{\text{vis}}^2},$$

run in *exclusive* mode: the clustering is stopped when exactly N jets remain, with N fixed per process to the expected number of final-state jets (2, 4 or 6; see `src/process_config.py`). Because N is imposed, there is no jet-multiplicity distribution to speak of and no radius to scan. Call:

`JetClustering::clustering_ee_kt(2, N, 1, 0)`

(`exclusive = 2` meaning “exactly cut jets”, `cut = N`, E -ordered output).

e^+e^- **Cambridge/Aachen, inclusive, radius scan.** Eq. (1) with $p = 0$, so d_{ij} is purely angular: $d_{ij} = (1 - \cos \theta_{ij}) / (1 - \cos R)$, independent of energy. Pairs are merged in order of increasing opening angle. Run inclusively, so the number of jets is an output, not an input. Scanned over $R = 0.4, 0.6, 0.8, 1.0, 1.2, 1.4$. Call:

`JetClustering::clustering_ee_genkt(R, 0, 0, 0, 0, 0)`

Method directories `PF_EECambridgeR04... PF_EECambridgeR14`.

e^+e^- k_T , **inclusive, radius scan.** Eq. (1) with $p = +1$: d_{ij} is weighted by the *softer* of the two energies, so soft particles are clustered first and jets grow from the soft radiation inwards. Same inclusive mode and same radius grid as the C/A scan, which makes the two directly comparable at matched R . Call:

`JetClustering::clustering_ee_genkt(R, 0, 0, 0, 0, 1)`

Method directories `PF_EEKtR04... PF_EEKtR14`. At equal R , k_T merges considerably more aggressively than C/A: at $R = 0.8$ in $Z(\rightarrow qq)H(\rightarrow WW \rightarrow qq\bar{q}\bar{q})$, 74% of events still yield 6 jets under k_T against 98% under C/A.

e^+e^- **anti- k_T , inclusive, radius scan.** Eq. (1) with $p = -1$: d_{ij} is weighted by the *inverse* of the harder energy, so hard particles cluster first and soft radiation is swept into the nearest hard core rather than forming jets of its own. Same inclusive mode and radius grid as the other two, making all three comparable at matched R . Call:

`JetClustering::clustering_ee_genkt(R, 0, 0, 0, 0, -1)`

Method directories `PF_EEAntiKtR04... PF_EEAntiKtR14`. First produced in September 2026; before that the repository contained no genuine anti- k_T run at all (see the naming history below).

Ordering of the three exponents. Merging becomes monotonically more aggressive as p increases. Measured at $R = 0.8$ in $Z(\rightarrow qq)H(\rightarrow WW \rightarrow qq\bar{q}\bar{q})$ over 10^4 events:

p	algorithm	events yielding 6 jets	fully-matched pass rate
-1	anti- k_T	99.8%	0.878
0	Cambridge/Aachen	98.4%	0.815
+1	k_T	74.4%	0.611
—	Durham (exclusive N)	100% by construction	0.866

Energy recovery (the “-ER” variants). Not a clustering algorithm but a post-processing step applied to the inclusive C/A output: the N leading jets are kept and every surplus jet is recombined into them, so the event ends with exactly N jets (`ZHfunctions::energy_recovery`, `src/histmaker_functions/functions.h`). It exists to recover the energy that inclusive clustering leaves in extra jets, and is available for all three exponents (`PF_E_recovery_EECambridgeR*`, `PF_E_recovery_EEKtR*`, `PF_E_recovery_EEAntiKtR*`).

Because it acts only where the clustering overproduces, its size is governed by N . Measured at $R = 0.4$ on $Z(\rightarrow \nu\nu)H(\rightarrow qq)$ ($N = 2$) it moves the m_H peak from 119.9 to 124.9 GeV, and makes the result independent of both the radius and the exponent — the N jets then contain every visible particle, so nothing is left for the jet definition to change. On the 6-jet light-flavour process at $R \geq 0.8$, by contrast, the clustering already yields $\leq N$ jets and energy recovery is exactly the identity.

CaloJets. Not clustered in this framework at all: the jets are read from the `CaloJetDurham` branch produced by Delphes, which applies the Durham algorithm to calorimeter towers rather than to particle-flow objects. Included as the “no particle flow” reference.

Hadron-collider anti- k_T . Available as `-jet-algorithm AK`, i.e. `clustering_antikt` (FastJet `antikt_algorithm`), with the usual p_T /rapidity measure

$$d_{ij} = \min\left(p_{T,i}^{-2}, p_{T,j}^{-2}\right) \frac{\Delta R_{ij}^2}{R^2}, \quad d_{iB} = p_{T,i}^{-2}.$$

It is not used in any of the production figures and is kept only for cross-checks: the p_T /rapidity measure is not the appropriate one at a lepton collider.

Summary

Label in figures	FastJet	p	Mode	Method directory
Durham / PFJets	<code>ee_kt</code>	+1	excl., N jets	<code>PF_Durham</code>
CaloJets	<code>ee_kt</code> (Delphes)	+1	excl., N jets	<code>CaloJets_Durham</code>
ee-C/A R	<code>ee_genkt</code>	0	inclusive	<code>PF_EECambridgeR*</code>
ee-C/A R -ER	<code>ee_genkt</code>	0	incl. + recovery	<code>PF_E_recovery_EECambridgeR*</code>
ee- k_T R	<code>ee_genkt</code>	+1	inclusive	<code>PF_EEKtR*</code>
ee-anti- k_T R	<code>ee_genkt</code>	−1	inclusive	<code>PF_EEAntiKtR*</code>

Naming history (important when reading older figures)

Until September 2026 the C/A scan was labelled “generalized e^+e^- anti- k_T ” and its output directories were named `PF_AntiKtR*`. That label was wrong. The clustering call was

```
JetClustering::clustering_ee_genkt(R, 0, 0, 0)
```

with only four arguments, whereas the constructor signature is

```
(radius, exclusive, cut, sorted, recombination=0, exponent=0.)
```

The exponent therefore defaulted to 0, so every one of those runs was Cambridge/Aachen and never anti- k_T . The histograms themselves are valid C/A data, so they were relabelled rather than re-run, and the directories renamed to `PF_EECambridgeR*`.

Genuine e^+e^- anti- k_T ($p = -1$) is now run separately, as `EEAKT` with output in `PF_EEAntiKtR*`. Note that `EEAK` (no trailing T) and `EEAKT` are different algorithms: the former is the historical spelling that ran $p = 0$ and survives only as a deprecated alias of `EECA`. Likewise `PF_AntiKtR*` (Cambridge/Aachen, legacy) must not be confused with `PF_EEAntiKtR*` (genuine anti- k_T).

Any figure or table showing “ee-AK”, “anti-kt” or `AntiKtR` should be read as e^+e^- Cambridge/Aachen.